TEKSTAC HANDS-ON

Spring Core Basics

1) Display Staff Details:

ApplicationConfig.java

```
package com.spring.app;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan; // <-- IMPORT
THIS
import org.springframework.context.annotation.Configuration;
import java.util.ArrayList;
import java.util.List;

//Use appropriate annotations
@Configuration
@ComponentScan(basePackages = "com.spring.app") // <-- ADD THIS LINE
public class ApplicationConfig {

    //Use appropriate annotation
    @Bean
    public Staff staff() {
        // FILL THE CODE HERE
        // Data from sample output
        return new Staff(1, "Ragul", "CSE", 9445543300L);
    }

    //Use appropriate annotation
    @Bean
    public Department department() {
        // FILL THE CODE HERE
        List<Staff> staffList = new ArrayList<>();
        staffList.add(staff()); // Injecting the staff bean
        // Data from sample output
        return new Department(123, staffList);
    }
}
```

Driver.java

```
package com.spring.app;

import
org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Driver {

   public static Department loadStaffDetails() {
      // FILL THE CODE HERE
      AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext(ApplicationConfig.class);
      Department department = context.getBean(Department.class);
      context.close(); // Close the context to release resources
      return department;
   }

   public static void main(String[] args) {
      // FILL THE CODE HERE
      Department department = loadStaffDetails();
      if (department != null) {
         department.displayStaffDetails();
      }
   }
}
```

Department.java

```
package com.spring.app;

import java.util.List;
import org.springframework.stereotype.Component;

//use appropriate annotation to make this class as component class
@Component
public class Department {

   private int departmentId;
   private List<Staff> staffs;

   public int getDepartmentId() {
      return departmentId;
```

```
    }

    public void setDepartmentId(int departmentId) {
        this.departmentId = departmentId;
    }

    public List<Staff> getStaffs() {
        return staffs;
    }

    public void setStaffs(List<Staff> staffs) {
        this.staffs = staffs;
    }

    /* Write a two argument constructor which accepts departmentId and list of
     staffs as the parameters */
    public Department(int departmentId, List<Staff> staffs) {
        this.departmentId = departmentId;
        this.staffs = staffs;
    }

    // Default constructor
    public Department() {
    }

    public void displayStaffDetails() {
        // FILL THE CODE HERE
        System.out.println("Staff Details:");
        for (Staff staff : staffs) {
            System.out.println("Staff Id:" + staff.getStaffId());
            System.out.println("Staff Name:" + staff.getStaffName());
            System.out.println("Contact Number:" + staff.getContactNo());
            System.out.println("Department Name:" + staff.getDepartmentName());
        }
        System.out.println("Department Id:" + getDepartmentId());
    }
}
```

Staff.java

```
package com.spring.app;

import org.springframework.stereotype.Component;
```

```
//Use appropriate annotation to make this class as component class
@Component
public class Staff {

    private int staffId;
    private String staffName;
    private String departmentName;
    private long contactNo;

    public int getStaffId() {
        return staffId;
    }

    public void setStaffId(int staffId) {
        this.staffId = staffId;
    }

    public String getStaffName() {
        return staffName;
    }

    public void setStaffName(String staffName) {
        this.staffName = staffName;
    }

    public String getDepartmentName() {
        return departmentName;
    }

    public void setDepartmentName(String departmentName) {
        this.departmentName = departmentName;
    }

    public long getContactNo() {
        return contactNo;
    }

    public void setContactNo(long contactNo) {
        this.contactNo = contactNo;
    }
```

```
    /* Write a four argument constructor which accepts staffId, staffName,
departmentName
     and contactNo as the parameters. */
    public Staff(int staffId, String staffName, String departmentName, long contactNo)
{
        this.staffId = staffId;
        this.staffName = staffName;
        this.departmentName = departmentName;
        this.contactNo = contactNo;
    }

    // Default constructor is good practice to have, though not strictly required by this
problem
    public Staff() {
    }
}
```

2) Display Book Details – Autowiring

Book.java

```
package com.spring.app;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

//fill the code
@Component
public class Book {

    //fill the code
    @Value("123")
    private int bookId;
    //fill the code
    @Value("Dennis M.Ritchie")
    private String bookAuthor;
    //fill the code
    @Value("Programming Using C")
    private String bookTitle;
    //fill the code
    @Value("280.0")
    private double price;
```

```
    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public int getBookId() {
        return bookId;
    }

    public void setBookId(int bookId) {
        this.bookId = bookId;
    }

    public String getBookAuthor() {
        return bookAuthor;
    }

    public void setBookAuthor(String bookAuthor) {
        this.bookAuthor = bookAuthor;
    }

    public String getBookTitle() {
        return bookTitle;
    }

    public void setBookTitle(String bookTitle) {
        this.bookTitle = bookTitle;
    }
}
```

Order.java

```
package com.spring.app;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

//fill the code
@Component
```

```
public class Order {

    //fill the code
    @Autowired
    private Book books;
    //fill the code
    @Value("2")
    private int quantity;

    public Book getBooks() {
        return books;
    }

    public void setBooks(Book books) {
        this.books = books;
    }

    public int getQuantity() {
        return quantity;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }

    public double calculateTotalPrice() {
        double totalPrice = 0.0;
        //fill the code
        totalPrice = books.getPrice() * this.quantity;
        return totalPrice;
    }

    public void displayOrderDetails() {
        //fill the code
        System.out.println("Invoice Details:");
        System.out.println("Book Id:" + books.getBookId());
        System.out.println("Book Title:" + books.getBookTitle());
        System.out.println("Author:" + books.getBookAuthor());
        System.out.println("Quantity:" + this.quantity);
        System.out.println("Price of a book:" + books.getPrice());
        System.out.println("Total Bill Amount:" + calculateTotalPrice());
    }
}
```

ApplicationConfig.java

```
package com.spring.app;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

//fill the code
@Configuration
@ComponentScan(basePackages = "com.spring.app")
public class ApplicationConfig {

}
```

Driver.java

```
package com.spring.app;

import
org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Driver {

   public static Order loadBookDetails() {
      //fill the code
      AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext(ApplicationConfig.class);
      Order order = context.getBean(Order.class);
      context.close();
      return order;
   }

   public static void main(String[] args) {
      //fill the code
      Order order = loadBookDetails();
      if (order != null) {
         order.displayOrderDetails();
      }
   }
}
```

3) **Gold Rate Calculation – Collections**

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>GoldRateCalculation</groupId>
    <artifactId>GoldRateCalculation</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <dependencies>

        <dependency>
            <groupId>org.springframework</groupId>
            <artifactId>spring-context</artifactId>
            <version>5.3.10</version>
        </dependency>

        <dependency>
            <groupId>jakarta.xml.bind</groupId>
            <artifactId>jakarta.xml.bind-api</artifactId>
            <version>3.0.0</version>
        </dependency>

        <dependency>
            <groupId>com.github.stefanbirkner</groupId>
            <artifactId>system-rules</artifactId>
            <version>1.16.0</version>
```

```
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.12</version>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>xmlunit</groupId>
        <artifactId>xmlunit</artifactId>
        <version>1.6</version>
        <scope>test</scope>
    </dependency>

</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.0.0-M1</version>
            <configuration>
                <testFailureIgnore>true</testFailureIgnore>
            </configuration>
        </plugin>
```

```
        </plugins>
    </build>

   <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>
</project>
```

Driver.java

```
package com.spring.app;

import java.util.Scanner;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.PropertySource;

@Configuration
@ComponentScan
@PropertySource("classpath:goldRateDetails.properties")
public class Driver {

    public static GoldRateInfo loadGoldRateDetails() {
        ApplicationContext context = new
AnnotationConfigApplicationContext(GoldRateInfo.class);
        return context.getBean(GoldRateInfo.class);
    }
```

```
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        GoldRateInfo goldRateInfo = loadGoldRateDetails();

        System.out.println("Enter the carat:");
        int carat = in.nextInt();
        System.out.println("Enter Total Grams:");
        double grams = in.nextDouble();

        double totalRate = goldRateInfo.calculateGoldRate(carat, grams);
        System.out.printf("Total Gold Rate is Rs:%.1f", totalRate);

        in.close();
    }
}
```

Goldrateinfo.java

```
package com.spring.app;

import java.util.Map;
import java.util.Scanner;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.PropertySource;
import org.springframework.stereotype.Component;
import org.springframework.beans.factory.annotation.Value;
```

```
@Configuration
@PropertySource("classpath:goldRateDetails.properties")
@Component
public class GoldRateInfo {

    @Value("#{${gold.rate}}")
    private Map<Integer, Double> rateInfo;

    public Map<Integer, Double> getRateInfo() {
        return rateInfo;
    }

    public void setRateInfo(Map<Integer, Double> rateInfo) {
        this.rateInfo = rateInfo;
    }

    public double calculateGoldRate(int goldCarat, double grams) {
        double totalRate = 0.0;
        if (rateInfo.containsKey(goldCarat)) {
            double ratePerGram = rateInfo.get(goldCarat);
            totalRate = ratePerGram * grams;
        }
        return totalRate;
    }
}
```

4) **Shipment-Item-Scope**

ApplicationConfig.java

```
package com.spring.app;

import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan("com.spring.app")
public class ApplicationConfig {

}
```

Driver.java

```
package com.spring.app;

import org.springframework.context.ApplicationContext;
import
org.springframework.context.annotation.AnnotationConfigApplicationContext;
import java.util.Scanner;

public class Driver {
   public static void main(String[] args) {
      ApplicationContext context = new
AnnotationConfigApplicationContext(ApplicationConfig.class);
      Scanner sc = new Scanner(System.in);

      // Get the first shipment bean
      System.out.println("Shipment Details1");
      Shipment shipment1 = context.getBean(Shipment.class);
      System.out.println("Enter the Item Name");
      shipment1.getItem().setItemName(sc.nextLine());
      System.out.println("Enter the Item Price");
      shipment1.getItem().setPrice(Double.parseDouble(sc.nextLine()));
      System.out.println("Enter the ShipmentId");
      shipment1.setShipmentId(sc.nextLine());
      System.out.println("Enter the Delivery Status");
      shipment1.setDeliveryStatus(sc.nextLine());

      // Get the second shipment bean
      System.out.println("Shipment Details2");
      Shipment shipment2 = context.getBean(Shipment.class);
      System.out.println("Enter the Item Name");
```

```
        shipment2.getItem().setItemName(sc.nextLine());
        System.out.println("Enter the Item Price");
        shipment2.getItem().setPrice(Double.parseDouble(sc.nextLine()));
        System.out.println("Enter the ShipmentId");
        shipment2.setShipmentId(sc.nextLine());
        System.out.println("Enter the Delivery Status");
        shipment2.setDeliveryStatus(sc.nextLine());

        sc.close();

        System.out.println("Delivery status of shipment ID: " +
shipment1.getShipmentId() + " is " + shipment1.getDeliveryStatus());
        System.out.println("Delivery status of shipment ID: " +
shipment2.getShipmentId() + " is " + shipment2.getDeliveryStatus());
    }
}
```

Item.Java

```
package com.spring.app;

import org.springframework.stereotype.Component;

@Component
public class Item {
    private String itemName;
    private double price;

    public String getItemName() {
        return itemName;
    }

    public void setItemName(String itemName) {
        this.itemName = itemName;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }
```

}

Shipment.java

```
package com.spring.app;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

@Component
@Scope("prototype")
public class Shipment {
   private String shipmentId;
   @Autowired
   private Item item;
   private String deliveryStatus;

   public String getDeliveryStatus() {
      return deliveryStatus;
   }

   public void setDeliveryStatus(String deliveryStatus) {
      this.deliveryStatus = deliveryStatus;
   }

   public String getShipmentId() {
      return shipmentId;
   }

   public void setShipmentId(String shipmentId) {
      this.shipmentId = shipmentId;
   }

   public Item getItem() {
      return item;
   }

   public void setItem(Item item) {
      this.item = item;
   }
}
```

5) DBConfig-SetterBasedInjection

Pom.xml

```
package com.cts.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.cts.bean.Product;
import com.cts.dao.ProductDaoImpl;

@Service
public class ProductService {

   // wire dependency

   @Autowired
   private ProductDaoImpl productDaoImpl;

   public Product getProductDetails(int product_code) {
      // add your code here
      return productDaoImpl.getProductDetails(product_code);
   }

   public double calculatePrice(int quantity, double product_price) {
      // add your code here
      return quantity * product_price;
   }
```

```
    public boolean validateProductCode(int product_code) {
        // add your code here
        return product_code > 999 && product_code <= 9999;
    }

    public boolean validateQuantity(int quantity) {
        // add your code here
        return quantity > 0;
    }
}
```

Main.java

```
package com.app.driver;

import com.spring.app.ApplicationConfig;
import com.spring.app.StudentDAO;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    public static void main(String[] args) {
        ApplicationContext context = new
AnnotationConfigApplicationContext(ApplicationConfig.class);
        StudentDAO studentDAO = context.getBean(StudentDAO.class);
        System.out.println(studentDAO.getDbConfig().getDriverName());
        System.out.println(studentDAO.getDbConfig().getUrl());
        System.out.println(studentDAO.getDbConfig().getUserName());
        System.out.println(studentDAO.getDbConfig().getPassword());
    }
```

```
}
```

ApplicationConfig.java

```
package com.spring.app;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

@Configuration
@ComponentScan(basePackages = "com.spring.app")
public class ApplicationConfig {

   @Bean
   public DBConfig dbConfig() {
      DBConfig dbConfig = new DBConfig();
      dbConfig.setDriverName("oracle.jdbc.driver.OracleDriver");
      dbConfig.setUrl("jdbc:oracle:thin:@localhost:1521:oracle");
      dbConfig.setUserName("alex");
      dbConfig.setPassword("alex@123");
      return dbConfig;
   }

   @Bean
   public StudentDAO studentDAO() {
      StudentDAO studentDAO = new StudentDAO();
      studentDAO.setDbConfig(dbConfig());
      return studentDAO;
   }
```

```
}
```

DBConfig.java

```
package com.spring.app;

public class DBConfig {
    private String driverName;
    private String url;
    private String userName;
    private String password;

    public String getDriverName() {
        return driverName;
    }

    public void setDriverName(String driverName) {
        this.driverName = driverName;
    }

    public String getUrl() {
        return url;
    }

    public void setUrl(String url) {
        this.url = url;
    }

    public String getUserName() {
        return userName;
```

```
    }

    public void setUserName(String userName) {
        this.userName = userName;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

StudentDAO.java

```
package com.spring.app;

public class StudentDAO {
    private DBConfig dbConfig;

    public DBConfig getDbConfig() {
        return dbConfig;
    }

    public void setDbConfig(DBConfig dbConfig) {
        this.dbConfig = dbConfig;
    }
}
```

6) **Account-Loan**

Loan.java

```
package com.spring.app;

import org.springframework.stereotype.Component;

// The @Component annotation is added here as per the error message.
// This registers Loan as a Spring bean.
@Component
public class Loan {
   // Private attributes for loan details
   private String loanType;
   private double loanAmount;

   // Default constructor is required by Spring
   public Loan() {
      System.out.println("Loan object created");
   }

   // Getter and setter methods for loan properties
   public String getLoanType() {
      return loanType;
   }

   public void setLoanType(String loanType) {
      this.loanType = loanType;
   }

   public double getLoanAmount() {
      return loanAmount;
   }

   public void setLoanAmount(double loanAmount) {
      this.loanAmount = loanAmount;
   }
}
```

Account.java

```
package com.spring.app;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;

// The Account class represents a bank account and its associated loan.
@Component
public class Account {

    // Private attributes for account details
    private int accNumber;
    private String accHolderName;
    private double accBalance;

    // The @Autowired annotation is placed on the loanInfo attribute
    // as per the error message. This is a form of field injection.
    @Autowired
    private Loan loanInfo;

    // No-arg constructor is often needed by Spring framework
    public Account() {
        System.out.println("Account object created");
    }

    // Getter and setter methods for account properties
    public int getAccNumber() {
        return accNumber;
    }

    public void setAccNumber(int accNumber) {
        this.accNumber = accNumber;
    }

    public String getAccHolderName() {
        return accHolderName;
    }

    public void setAccHolderName(String accHolderName) {
        this.accHolderName = accHolderName;
    }

    public double getAccBalance() {
```

```
        return accBalance;
    }

    public void setAccBalance(double accBalance) {
        this.accBalance = accBalance;
    }

    public Loan getLoanInfo() {
        return loanInfo;
    }

    // The setter method is no longer needed for autowiring but is kept
    // for general good practice.
    public void setLoanInfo(Loan loanInfo) {
        this.loanInfo = loanInfo;
    }
}
```

ApplicationConfig.java

```
package com.spring.app;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;

// @Configuration indicates that this class provides bean definitions.
@Configuration
// @ComponentScan is used here so Spring can find and register the
@Component classes
// like Account and Loan.
@ComponentScan(basePackages = "com.spring.app")
public class ApplicationConfig {

    // The @Bean method defines the Loan bean.
    // This is now a separate bean method as per the error.
    @Bean
    public Loan loan() {
        Loan loan = new Loan();
        loan.setLoanType("HomeLoan");
        loan.setLoanAmount(150000.0);
        return loan;
    }
```

```
    // This @Bean method defines the Account bean.
    // Spring will automatically inject the Loan bean defined above
    // using the @Autowired annotation in the Account class.
    @Bean
    public Account account() {
        Account account = new Account();
        account.setAccNumber(335647852);
        account.setAccHolderName("Vanitha");
        account.setAccBalance(250000.0);
        return account;
    }
}
```

Driver.java

```
package com.spring.app;

import org.springframework.context.ApplicationContext;
import
org.springframework.context.annotation.AnnotationConfigApplicationContext;

// The Driver class contains the main method to run the application.
public class Driver {

    // This method loads the Spring context and retrieves the Account bean.
    public static Account loadAccount() {
        // Create an ApplicationContext using the
AnnotationConfigApplicationContext,
        // which loads bean definitions from the ApplicationConfig class.
        ApplicationContext context = new
AnnotationConfigApplicationContext(ApplicationConfig.class);

        // Retrieve the Account bean from the context.
        return context.getBean(Account.class);
    }

    public static void main(String[] args) {
        // Invoke the loadAccount method to get the fully initialized Account object.
        Account account = loadAccount();

        // Retrieve the Loan object from the Account object.
```

```
            Loan loan = account.getLoanInfo();

            // Display the account and loan details as per the sample output.
            System.out.println("Account number:" + account.getAccNumber());
            System.out.println("Account holder name:" +
        account.getAccHolderName());
            System.out.println("Balance:" + account.getAccBalance());
            System.out.println("Loan type:" + loan.getLoanType());
            System.out.println("Loan amount:" + loan.getLoanAmount());
          }
        }
```

7) Azure – Horizon Resort

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>AzureHorizonResort</groupId>
<artifactId>AzureHorizonResort</artifactId>
<version>0.0.1-SNAPSHOT</version>

  <dependencies>
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>5.3.10</version>
</dependency>
<dependency>
<groupId>jakarta.xml.bind</groupId>
<artifactId>jakarta.xml.bind-api</artifactId>
<version>3.0.0</version>
</dependency>
```

```
<dependency>
<groupId>com.github.stefanbirkner</groupId>
<artifactId>system-rules</artifactId>
<version>1.16.0</version>
<scope>test</scope>
</dependency>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.12</version>
<scope>test</scope>
</dependency>
<dependency>
<groupId>xmlunit</groupId>
<artifactId>xmlunit</artifactId>
<version>1.6</version>
<scope>test</scope>
</dependency>
</dependencies>

    <build>
<plugins>
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>3.0.0-M1</version>
<configuration>
<testFailureIgnore>true</testFailureIgnore>
</configuration>
```

```
</plugin>
</plugins>
</build>

   <properties>
<maven.compiler.source>17</maven.compiler.source>
<maven.compiler.target>17</maven.compiler.target>
</properties>
</project>
```

ApplicationConfig.java

```
package com.hotel;

import java.text.ParseException;
import java.text.SimpleDateFormat;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.PropertySource;

import com.hotel.model.Reservation;
import com.hotel.service.ReservationService;

@Configuration
@ComponentScan(basePackages = "com.hotel")
@PropertySource("classpath:details.properties")
public class ApplicationConfig {
```

```
    public static void main(String[] args) throws ParseException {
        ApplicationContext context =
                new AnnotationConfigApplicationContext(ApplicationConfig.class,
ConversionConfig.class);

        ReservationService service = context.getBean(ReservationService.class);
        Reservation reservation = context.getBean(Reservation.class);

        reservation.setCustomerName("John Doe");
        reservation.setPhoneNumber(9876543210L);
        reservation.setRoomType("Deluxe");
        reservation.setCheckInDate(new
SimpleDateFormat("dd/MM/yyyy").parse("01/09/2025"));
        reservation.setCheckOutDate(new
SimpleDateFormat("dd/MM/yyyy").parse("05/09/2025"));
        reservation.setNumOfGuests(2);

        service.bookReservation(reservation);
    }
}
```

ConversionConfig.java

```
package com.hotel;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.convert.converter.Converter;
import org.springframework.core.convert.support.DefaultConversionService;
```

```
import org.springframework.core.convert.support.GenericConversionService;
import java.util.HashMap;
import java.util.Map;

@Configuration
public class ConversionConfig {

    @Bean
    public GenericConversionService conversionService() {
        GenericConversionService conversionService = new DefaultConversionService();
        conversionService.addConverter(new StringToMapConverter());
        return conversionService;
    }

    // Custom converter for converting String to Map
    private static class StringToMapConverter implements Converter<String, Map<String, Double>> {
        @Override
        public Map<String, Double> convert(String source) {
            Map<String, Double> map = new HashMap<>();
            String[] pairs = source.substring(1, source.length() - 1).split(", ");
            for (String pair : pairs) {
                String[] keyValue = pair.split("=");
                map.put(keyValue[0], Double.valueOf(keyValue[1]));
            }
            return map;
        }
    }
}
```

Hotel.java

```
package com.hotel.model;

import java.util.Map;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class Hotel {

    @Value("${hotel.name}")
    private String hotelName;

    @Value("${hotel.location}")
    private String location;

    @Value("${hotel.roomRates}")
    private Map<String, Double> roomRates;

    public String getHotelName() {
        return hotelName;
    }

    public void setHotelName(String hotelName) {
        this.hotelName = hotelName;
    }

    public String getLocation() {
```

```
        return location;
    }

    public void setLocation(String location) {
        this.location = location;
    }

    public Map<String, Double> getRoomRates() {
        return roomRates;
    }

    public void setRoomRates(Map<String, Double> roomRates) {
        this.roomRates = roomRates;
    }
}
```

Reservation.java

```
package com.hotel.model;

import java.util.Date;
import org.springframework.stereotype.Component;

@Component
public class Reservation {

    private String customerName;
    private Date checkInDate;
    private Date checkOutDate;
    private Long phoneNumber;
```

```
private int numOfGuests;
private String roomType;

public String getCustomerName() {
    return customerName;
}

public void setCustomerName(String customerName) {
    this.customerName = customerName;
}

public Date getCheckInDate() {
    return checkInDate;
}

public void setCheckInDate(Date checkInDate) {
    this.checkInDate = checkInDate;
}

public Date getCheckOutDate() {
    return checkOutDate;
}

public void setCheckOutDate(Date checkOutDate) {
    this.checkOutDate = checkOutDate;
}

public Long getPhoneNumber() {
    return phoneNumber;
```

```
    }

    public void setPhoneNumber(Long phoneNumber) {
        this.phoneNumber = phoneNumber;
    }

    public int getNumOfGuests() {
        return numOfGuests;
    }

    public void setNumOfGuests(int numOfGuests) {
        this.numOfGuests = numOfGuests;
    }

    public String getRoomType() {
        return roomType;
    }

    public void setRoomType(String roomType) {
        this.roomType = roomType;
    }
}
```

ReservationService.java

```
package com.hotel.service;

import java.text.ParseException;
import java.text.SimpleDateFormat;
```

```
import java.util.Date;
import java.util.Scanner;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.hotel.model.Hotel;
import com.hotel.model.Reservation;

@Service
public class ReservationService {

   @Autowired
   private Hotel hotel;

   public void setHotel(Hotel hotel) {
      this.hotel = hotel;
   }

   public void bookReservation(Reservation reservation) throws ParseException {
      Scanner sc = new Scanner(System.in);
      SimpleDateFormat sdf = new SimpleDateFormat("dd-MM-yyyy");

      System.out.println("Enter the Customer Name");
      reservation.setCustomerName(sc.nextLine());

      System.out.println("Enter the Phone Number");
      reservation.setPhoneNumber(Long.parseLong(sc.nextLine()));
```

```
        System.out.println("Enter the Check-In Date <dd-MM-yyyy>");
        Date checkIn = sdf.parse(sc.nextLine());
        reservation.setCheckInDate(checkIn);

        System.out.println("Enter the Check-Out Date <dd-MM-yyyy>");
        Date checkOut = sdf.parse(sc.nextLine());
        reservation.setCheckOutDate(checkOut);

        System.out.println("Enter the Total Number of Guests");
        reservation.setNumOfGuests(Integer.parseInt(sc.nextLine()));

        System.out.println("Enter the Room Type");
        reservation.setRoomType(sc.nextLine());

        if (checkAvailability(checkIn, checkOut)) {
            double cost = calculateBookingCost(reservation.getRoomType(), checkIn,
checkOut);
            System.out.println("Total Booking Cost is: $" + cost);
        } else {
            System.out.println("Invalid Details");
        }
    }

    public double calculateBookingCost(String roomType, Date checkInDate, Date
checkOutDate) {
        Double roomRate = hotel.getRoomRates().get(roomType);
        if (roomRate == null) {
            return 0.0;
        }
        long diff = checkOutDate.getTime() - checkInDate.getTime();
```

```
        int days = (int) (diff / (1000 * 60 * 60 * 24));
        return roomRate * days;
    }

    public boolean checkAvailability(Date checkInDate, Date checkOutDate) {
        // Simple logic: check-out must be after check-in
        return checkOutDate.after(checkInDate);
    }
}
```

Details.properties

```
hotel.name=Azure Horizon Resort HOTELS
hotel.location=Chennai
hotel.roomRates={Standard=100.0, Deluxe=150.0, Suite=200.0}
```

8) Hospital Management

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
<groupId>HospitalManagement</groupId>
<artifactId>HospitalManagement</artifactId>
<version>0.0.1-SNAPSHOT</version>

 <dependencies>
   <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>5.3.10</version>
      </dependency>

      <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.12</version>
        <scope>test</scope>
    </dependency>

      <dependency>
        <groupId>xmlunit</groupId>
        <artifactId>xmlunit</artifactId>
        <version>1.6</version>
        <scope>test</scope>
    </dependency>

    <dependency>
        <groupId>com.github.stefanbirkner</groupId>
        <artifactId>system-rules</artifactId>
```

```
            <version>1.16.0</version>
            <scope>test</scope>
        </dependency>
      </dependencies>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.target>17</maven.compiler.target>
        <maven.compiler.source>17</maven.compiler.source>
    </properties>

</project>
```

App.java

```
package com.spring.app;

import java.util.Scanner;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class App {
    public static void main(String[] args) {



        ApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);
        HeadHospital hospital = null;
        Scanner scanner = new Scanner(System.in);
```

```
        System.out.println("Select location:\n1.New York\n2.Los Angeles");
        int choice = scanner.nextInt();

        if(choice == 1){
          // FILL THE CODE HERE
          hospital = context.getBean(NewYorkHospital.class);
        } else if(choice == 2){
           hospital = context.getBean(LosAngelesHospital.class);
           // FILL THE CODE HERE
        } else {
           System.out.println("Wrong Choice");
           return;
        }
        hospital.doDocumentVerification();
        hospital.provideTreatment();
    }
}
```

AppConfig.java

```
package com.spring.app;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;

@Configuration
@ComponentScan("com.spring.app")

//Use appropriate annotation
```

```
public class AppConfig {

    //Use appropriate annotation
    @Bean
    public Document newYorkDocument() {

        Document document = new Document();
        document.setName("Alice");
        document.setIdProof("Insurance");
        document.setCondition("Flu");

        return document;
    }

    //Use appropriate annotation
    @Bean
    public Document losAngelesDocument() {

        Document document = new Document();
        document.setName("Bob");
        document.setIdProof("ID Card");
        document.setCondition("Sprain");

        return document;
    }
}
```

Document.java

```
package com.spring.app;
```

```
import org.springframework.stereotype.Component;

@Component
public class Document {

    private String name;
    private String idProof;
    private String condition;

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getIdProof() {
        return idProof;
    }
    public void setIdProof(String idProof) {
        this.idProof = idProof;
    }
    public String getCondition() {
        return condition;
    }
    public void setCondition(String condition) {
        this.condition = condition;
    }
}
```

HeadHospital.java

```
package com.spring.app;

public interface HeadHospital {

   public void doDocumentVerification();

   public void provideTreatment();
}
```

LosAngelesHospital.java

```
package com.spring.app;
import org.springframework.stereotype.Component;
import org.springframework.beans.factory.annotation.Autowired;
//use appropriate annotation to make this class as component class
@Component
public class LosAngelesHospital implements HeadHospital {

   private Document losAngelesDocument;

   //use appropriate annotation
   @Autowired
   public LosAngelesHospital(Document losAngelesDocument) {
      super();
      this.losAngelesDocument = losAngelesDocument;
   }
```

```
    // FILL THE CODE HERE
    @Override
    public void doDocumentVerification(){
        System.out.println("Document verification done
using"+losAngelesDocument.getIdProof());


    }
    @Override
    public void provideTreatment(){
        System.out.println("Treatment is in progress for
"+losAngelesDocument.getName()+" with condition
"+losAngelesDocument.getCondition()+" at Los Angeles Hospital");
    }
}
```

NewYorkHospital.java

```
package com.spring.app;
import org.springframework.stereotype.Component;
import org.springframework.beans.factory.annotation.Autowired;
//use appropriate annotation to make this class as component class
@Component
public class NewYorkHospital implements HeadHospital{


    private Document newYorkDocument;


    //use appropriate annotation
     @Autowired
    public NewYorkHospital(Document newYorkDocument) {
        super();
        this.newYorkDocument = newYorkDocument;
```

```
    }
    @Override
    public void doDocumentVerification(){
        System.out.println("Document verification done using
"+newYorkDocument.getIdProof());


    }
    @Override
    public void provideTreatment(){
        System.out.println("Treatment is in progress for "+newYorkDocument.getName()+"
with condition "+newYorkDocument.getCondition()+" at New York Hospital");
    }
   // FILL THE CODE HERE
}
```

9) PatientManagement

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>PatientManagement</groupId>
  <artifactId>PatientManagement</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>PatientManagement</name>
  <description>PatientManagement</description>


    <dependencies>
```

```
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>5.3.10</version>
       </dependency>



    </dependencies>

   <properties>
      <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
      <maven.compiler.target>17</maven.compiler.target>
      <maven.compiler.source>17</maven.compiler.source>
   </properties>

</project>
```

AppConfig.java

```
package com.cts.patient.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.PropertySource;
import org.springframework.context.support.PropertySourcesPlaceholderConfigurer;
import org.springframework.core.io.ClassPathResource;
import org.springframework.beans.factory.annotation.Value;
import com.cts.patient.model.Patient;
```

```
import com.cts.patient.service.PatientService;

//fill the code
@Configuration
@PropertySource("classpath:patient_details.properties")
public class AppConfig {

    //fill the code
    @Bean
    public static PropertySourcesPlaceholderConfigurer propertyConfig(){
        return new PropertySourcesPlaceholderConfigurer();
    }
    @Bean
    public Patient patient(
        @Value("${pid}") String pid,
        @Value("${pname}") String pname,
        @Value("${disease}") String disease,
        @Value("${sex}") String sex,
        @Value("${admit_status}") String admitStatus,
        @Value("${age}") Integer age
    ){
        Patient patient = new Patient();
        patient.setPid(pid);
        patient.setPname(pname);
        patient.setDisease(disease);
        patient.setSex(sex);
        patient.setAdmit_status(admitStatus);
        patient.setAge(age);
        return patient;
```

```
    }
    @Bean
    public PatientService patientService(){
        PatientService service = new PatientService();
        service.setPatient(patient(null,null,null,null,null,null));
        return service;
    }
}
```

Main.java

```
package com.cts.patient;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import com.cts.patient.config.AppConfig;
import com.cts.patient.service.*;

public class Main {

    public static void main(String[] args) {
    //fill the code
   ApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);
   PatientService service = context.getBean(PatientService.class);
   service.getPatientDetails();
    }

}
```

Patient.java

```
package com.cts.patient.model;


import org.springframework.beans.factory.annotation.Value;


public class Patient {


    //fill the code
    private String pid;
    //fill the code
    private String pname;
    //fill the code
    private String disease;
    //fill the code
    private String sex;
    //fill the code
    private String admit_status;
    //fill the code
    private Integer age;


    public String getPid() {
            return pid;
    }


    public void setPid(String pid) {
            this.pid = pid;
    }


    public String getPname() {
```

```
        return pname;
}

public void setPname(String pname) {
        this.pname = pname;
}

public String getDisease() {
        return disease;
}

public void setDisease(String disease) {
        this.disease = disease;
}

public String getSex() {
        return sex;
}

public void setSex(String sex) {
        this.sex = sex;
}

public String getAdmit_status() {
        return admit_status;
}

public void setAdmit_status(String admit_status) {
```

```
        this.admit_status = admit_status;
    }

    public Integer getAge() {
        return age;
    }

    public void setAge(Integer age) {
        this.age = age;
    }

}
```

PatienceService.java

```
package com.cts.patient.service;

import org.springframework.beans.factory.annotation.Autowired;

import com.cts.patient.model.Patient;

public class PatientService {

    //fill the code
    private Patient patient;
    public void setPatient(Patient patient){
       this.patient = patient;
    }
   public Patient getPatient(){
      return patient;
```

```
    }
     public void getPatientDetails() {
                //fill the code
                System.out.println("Patient Id : " + patient.getPid());
                System.out.println("Patient Name : " + patient.getPname());
                System.out.println("Disease : " + patient.getDisease());
                System.out.println("Sex : " + patient.getSex());
                System.out.println("Admit Status : " + patient.getAdmit_status());
                System.out.println("Age : " + patient.getAge());



     }

}
```

10) Loan Pro

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>LendPro</groupId>
<artifactId>LendPro</artifactId>
<version>0.0.1-SNAPSHOT</version>
<dependencies>
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-context</artifactId>
<version>5.3.10</version>
```

```
</dependency>
<dependency>
<groupId>junit</groupId>
<artifactId>junit</artifactId>
<version>4.12</version>
<scope>test</scope>
</dependency>
<dependency>
<groupId>xmlunit</groupId>
<artifactId>xmlunit</artifactId>
<version>1.6</version>
<scope>test</scope>
</dependency>
<dependency>
<groupId>com.github.stefanbirkner</groupId>
<artifactId>system-rules</artifactId>
<version>1.16.0</version>
<scope>test</scope>
</dependency>
</dependencies>
<properties>
<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
<maven.compiler.target>17</maven.compiler.target>
<maven.compiler.source>17</maven.compiler.source>
</properties>
</project>
```

Driver.java

```
package com.spring.app;
```

```
import java.util.Scanner;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Driver {
    public static void main(String[] args) {
        ApplicationContext context = new ClassPathXmlApplicationContext("beans.xml");
// assuming XML file is named beans.xml based on previous problems
        SmartLoan smartLoan = (SmartLoan) context.getBean("smartLoan");

        Scanner scanner = new Scanner(System.in);

        System.out.println("Welcome to Loan Processing System");
        System.out.println("Customer Name: " + smartLoan.getCustomerName());
        System.out.println("Customer ID: " + smartLoan.getCustomerId());

        System.out.println("Enter loan amount");
        double amount = scanner.nextDouble();

        System.out.println("Enter loan tenure in months");
        int tenure = scanner.nextInt();

        System.out.println("Enter loan type");
        String type = scanner.next();

        double emi = smartLoan.calculateEMI(amount, tenure, type);

        if (emi == -1.0) {
```

```
            System.out.println("Invalid Input");
        } else {
            System.out.printf("Your EMI for %d months will be $%.2f", tenure, emi);
        }

        scanner.close();
    }
}
```

Loan.java

```
package com.spring.app;

import java.util.List;

public abstract class Loan {

    private int customerId;
    private String customerName;

    public Loan() {

    }

    public Loan(int customerId, String customerName) {
        this.customerId = customerId;
        this.customerName = customerName;
    }

    public int getCustomerId() {
```

```
        return customerId;
    }

    public void setCustomerId(int customerId) {
        this.customerId = customerId;
    }

    public String getCustomerName() {
        return customerName;
    }

    public void setCustomerName(String customerName) {
        this.customerName = customerName;
    }

    public abstract double calculateEMI(double loanAmount, int tenure, String loanType);
}
```

SmartLoan.java

```
package com.spring.app;

import java.text.DecimalFormat;
import java.util.Map;

public class SmartLoan extends Loan {

    private Map<String, Double> interestRatesMap;
```

```
    public SmartLoan(int customerId, String customerName, Map<String, Double>
interestRatesMap) {
        super(customerId, customerName);
        this.interestRatesMap = interestRatesMap;
    }

    public Map<String, Double> getInterestRatesMap() {
        return interestRatesMap;
    }

    public void setInterestRatesMap(Map<String, Double> interestRatesMap) {
        this.interestRatesMap = interestRatesMap;
    }

    @Override
    public double calculateEMI(double loanAmount, int tenure, String loanType) {
        if (loanAmount <= 0 || tenure <= 0 || !interestRatesMap.containsKey(loanType)) {
            return -1.0;
        }

        double annualRate = interestRatesMap.get(loanType);
        double monthlyRate = annualRate / 12;
        double numerator = loanAmount * monthlyRate * Math.pow(1 + monthlyRate,
tenure);
        double denominator = Math.pow(1 + monthlyRate, tenure) - 1;
        double emi = numerator / denominator;

        DecimalFormat df = new DecimalFormat("#.##");
        return Double.parseDouble(df.format(emi));
    }
```

}

Beans.xml

```
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xmlns:context="http://www.springframework.org/schema/context"
      xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         http://www.springframework.org/schema/beans/spring-beans.xsd
         http://www.springframework.org/schema/context
         http://www.springframework.org/schema/context/spring-context.xsd">


   <bean
class="org.springframework.beans.factory.config.PropertyPlaceholderConfigurer">
       <property name="location" value="classpath:loanTypes.properties"/>
   </bean>
   <bean id="loan" class="com.spring.app.Loan" abstract="true"/>


   <bean id="smartLoan" class="com.spring.app.SmartLoan" parent="loan">
      <constructor-arg value="12345"/>
      <constructor-arg value="John"/>
      <constructor-arg>
         <map>
            <entry key="personalLoan" value="${personalLoan}"/>
            <entry key="homeLoan" value="${homeLoan}"/>
            <entry key="carLoan" value="${carLoan}"/>
         </map>
      </constructor-arg>
   </bean>
```

</beans>

LoanTypesProperties

personalLoan=0.085

homeLoan=0.075

carLoan=0.09

11) Billing Software Application

CreateScripts.sql

```
create database if not exists eshopping;
use eshopping;
drop table if exists product;

create table product
(
   product_code int primary key,
   product_name varchar(50) null,
   product_category varchar(50) null,
   product_desc varchar(50) null,
   product_price decimal(8,2)  null
);


 insert into product values(1001,'mobile','electronics','smart phone',35000);
 insert into product values(1002,'LEDTV','electronics','TV',45000);
 insert into product values(1003,'Teddy','toys','soft toy',800);
 insert into product values(1004,'Pencil','stationary','A pack of 12 pencils',80);
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>ProductBillingApplication</groupId>
  <artifactId>ProductBillingApplication</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <dependencies>
      <dependency>
          <groupId>org.springframework</groupId>
          <artifactId>spring-context</artifactId>
          <version>5.3.10</version>
      </dependency>




      <dependency>
          <groupId>com.github.stefanbirkner</groupId>
          <artifactId>system-rules</artifactId>
          <version>1.16.0</version>
          <scope>test</scope>
      </dependency>

      <dependency>
          <groupId>org.aspectj</groupId>
          <artifactId>aspectjrt</artifactId>
```

```
    <version>1.6.11</version>
</dependency>

<dependency>
    <groupId>org.aspectj</groupId>
    <artifactId>aspectjweaver</artifactId>
    <version>1.6.11</version>
</dependency>

<dependency>
    <groupId>log4j</groupId>
    <artifactId>log4j</artifactId>
    <version>1.2.17</version>
</dependency>




<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.26</version>
    <scope>runtime</scope>
</dependency>

<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-jdbc</artifactId>
    <version>5.3.10</version>
</dependency>
```

```
</dependencies>

   <build>
       <plugins>
         <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.0.0-M1</version>
            <configuration>
               <testFailureIgnore>true</testFailureIgnore>
            </configuration>
         </plugin>
       </plugins>
   </build>

<properties>
   <maven.compiler.source>17</maven.compiler.source>
   <maven.compiler.target>17</maven.compiler.target>
</properties>

</project>
```

AppConfiguration.java

```
package com.cts;

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;
```

```
import javax.sql.DataSource;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

@Configuration
@EnableTransactionManagement
@ComponentScan("com.cts")
public class AppConfiguration {
   @Bean
   public DataSource mysqlDataSource() {
      DriverManagerDataSource dataSource = new DriverManagerDataSource();
      ClassLoader classLoader = Thread.currentThread().getContextClassLoader();
      InputStream input = classLoader.getResourceAsStream("connection.properties");
      Properties properties = new Properties();
      try {
         properties.load(input);
         dataSource.setDriverClassName(properties.getProperty("driver"));
         dataSource.setUrl(properties.getProperty("url"));
         dataSource.setUsername(properties.getProperty("username"));
         dataSource.setPassword(properties.getProperty("password"));
      } catch (IOException e) {
         e.printStackTrace();
```

```
        }
        return dataSource;
    }

    @Bean
    public JdbcTemplate jdbcTemplate() {
        JdbcTemplate template = new JdbcTemplate(mysqlDataSource());
        return template;
    }

    @Bean
    public PlatformTransactionManager txManager() {
        return new DataSourceTransactionManager(mysqlDataSource());
    }

}
```

Product.java

```
package com.cts.bean;

public class Product {

    private int product_code;
    private String product_name;
    private String product_category;
    private String product_desc;
    private double product_price;

    public Product() {
```

```
    // TODO Auto-generated constructor stub
  }

  public Product(int product_code, String product_name, String product_category, String
product_desc,
      double product_price) {
    super();
    this.product_code = product_code;
    this.product_name = product_name;
    this.product_category = product_category;
    this.product_desc = product_desc;
    this.product_price = product_price;
  }

  public int getProduct_code() {
    return product_code;
  }

  public void setProduct_code(int product_code) {
    this.product_code = product_code;
  }

  public String getProduct_name() {
    return product_name;
  }

  public void setProduct_name(String product_name) {
    this.product_name = product_name;
  }
```

```
    public String getProduct_category() {
        return product_category;
    }

    public void setProduct_category(String product_category) {
        this.product_category = product_category;
    }

    public String getProduct_desc() {
        return product_desc;
    }

    public void setProduct_desc(String product_desc) {
        this.product_desc = product_desc;
    }

    public double getProduct_price() {
        return product_price;
    }

    public void setProduct_price(double product_price) {
        this.product_price = product_price;
    }

    @Override
    public String toString() {
        return "product_code=" + product_code + ", product_name=" + product_name + ",
product_category="
```

```
            + product_category + ", product_description=" + product_desc + ",
product_price=" + product_price;
    }

}
```

BillingSoftwareApplication.java

```
package com.cts.bean;

public class Product {

    private int product_code;
    private String product_name;
    private String product_category;
    private String product_desc;
    private double product_price;

    public Product() {
        // TODO Auto-generated constructor stub
    }

    public Product(int product_code, String product_name, String product_category, String
product_desc,
            double product_price) {
        super();
        this.product_code = product_code;
        this.product_name = product_name;
        this.product_category = product_category;
        this.product_desc = product_desc;
```

```
    this.product_price = product_price;
}

public int getProduct_code() {
    return product_code;
}

public void setProduct_code(int product_code) {
    this.product_code = product_code;
}

public String getProduct_name() {
    return product_name;
}

public void setProduct_name(String product_name) {
    this.product_name = product_name;
}

public String getProduct_category() {
    return product_category;
}

public void setProduct_category(String product_category) {
    this.product_category = product_category;
}

public String getProduct_desc() {
    return product_desc;
```

```
    }

    public void setProduct_desc(String product_desc) {
        this.product_desc = product_desc;
    }

    public double getProduct_price() {
        return product_price;
    }

    public void setProduct_price(double product_price) {
        this.product_price = product_price;
    }

    @Override
    public String toString() {
        return "product_code=" + product_code + ", product_name=" + product_name + ",
product_category="
                + product_category + ", product_description=" + product_desc + ",
product_price=" + product_price;
    }

}
```

ProductDAO.java

```
package com.cts.dao;
```

```
import com.cts.bean.Product;

public interface ProductDao {

    Product getProductDetails(int product_code);
}
```

ProductDAOImpl.java

```
package com.cts.dao;

import java.sql.ResultSet;
import java.sql.SQLException;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.stereotype.Repository;

import com.cts.bean.Product;

@Repository
public class ProductDaoImpl implements ProductDao {

    // wire dependency

    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Override
```

```
    public Product getProductDetails(int product_code) {
        String query = "select * from product where product_code = ?";

        return jdbcTemplate.queryForObject(query, new Object[] { product_code }, new
RowMapper<Product>() {
            @Override
            public Product mapRow(ResultSet rs, int rowNum) throws SQLException {
                return new Product(rs.getInt("product_code"), rs.getString("product_name"),
                        rs.getString("product_category"), rs.getString("product_desc"),
rs.getDouble("product_price"));
            }
        });

    }
}
```

BillingException.java

```
package com.cts.service;

public class BillingException extends Exception {

    public BillingException() {
        super();
    }

    public BillingException(String arg0, Throwable arg1, boolean arg2, boolean arg3) {
        super(arg0, arg1, arg2, arg3);
    }
```

```
    public BillingException(String arg0, Throwable arg1) {
        super(arg0, arg1);
    }

    public BillingException(String arg0) {
        super(arg0);
    }

    public BillingException(Throwable arg0) {
        super(arg0);
    }

}
```

ProductService.java

```
package com.cts.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.cts.bean.Product;
import com.cts.dao.ProductDaoImpl;

@Service
public class ProductService {

    // wire dependency

    @Autowired
```

```
    private ProductDaoImpl productDaoImpl;

    public Product getProductDetails(int product_code) {
        // add your code here
        return productDaoImpl.getProductDetails(product_code);
    }

    public double calculatePrice(int quantity, double product_price) {
        // add your code here
        return quantity * product_price;
    }

    public boolean validateProductCode(int product_code) {
        // add your code here
        return product_code > 999 && product_code <= 9999;
    }

    public boolean validateQuantity(int quantity) {
        // add your code here
        return quantity > 0;
    }
}
```

12) EBill

Create-schema.sql

```
drop database if exists egov;
```

```
create database egov;
use egov;

drop table if exists ebill;
create table ebill (
   bill_number int primary key auto_increment,
   consumer_id varchar(50) not null,
   billing_month date not null,
   reading int not null,
   unit int not null,
   amount decimal(8,2) not null
) ENGINE=InnoDB AUTO_INCREMENT=100;

insert into ebill (consumer_id, billing_month, reading, unit, amount) values
('01-001-001-001','2020-04-01',10000,0,0.00),
('01-001-001-001','2020-05-01',10130,130,650.00),
('01-001-001-001','2020-06-01',10470,340,1700.00),
('01-001-001-001','2020-07-01',10840,370,1850.00);
```

Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>org.spring.Electricitybill</groupId>
    <artifactId>Electricitybill</artifactId>
    <version>0.0.1-SNAPSHOT</version>
```

```
<packaging>jar</packaging>

<name>Electricitybill</name>
<url>http://maven.apache.org</url>

<properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>

<dependencies>
        <dependency>
                <groupId>org.springframework</groupId>
                <artifactId>spring-context</artifactId>
                <version>5.3.10</version>
        </dependency>
        <dependency>
                <groupId>org.springframework</groupId>
                <artifactId>spring-jdbc</artifactId>
                <version>5.3.10</version>
        </dependency>

        <dependency>
                <groupId>mysql</groupId>
                <artifactId>mysql-connector-java</artifactId>
                <version>8.0.26</version>
                <scope>runtime</scope>
        </dependency>

        <dependency>
```

```
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-log4j12</artifactId>
            <version>1.7.30</version>
            <scope>runtime</scope>
        </dependency>

        <dependency>
            <groupId>org.aspectj</groupId>
            <artifactId>aspectjrt</artifactId>
            <version>1.9.6</version>
        </dependency>
        <dependency>
            <groupId>org.aspectj</groupId>
            <artifactId>aspectjweaver</artifactId>
            <version>1.9.6</version>
        </dependency>

        <dependency>
            <groupId>commons-logging</groupId>
            <artifactId>commons-logging</artifactId>
            <version>1.2</version>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
```

```
                <version>3.8.1</version>
                <configuration>
                    <source>1.8</source>
                    <target>1.8</target>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

AppConfiuration.java

```
package com.cts.handson;

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

import javax.sql.DataSource;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.datasource.DataSourceTransactionManager;
import org.springframework.jdbc.datasource.DriverManagerDataSource;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.annotation.EnableTransactionManagement;

@Configuration
```

```
@EnableTransactionManagement
@ComponentScan("com.cts.handson")
public class AppConfiguration {
   @Bean
   public DataSource mysqlDataSource() {
      DriverManagerDataSource dataSource = new DriverManagerDataSource();
      ClassLoader classLoader = Thread.currentThread().getContextClassLoader();
      InputStream input = classLoader.getResourceAsStream("connection.properties");
      Properties properties = new Properties();
      try {
         properties.load(input);
         dataSource.setDriverClassName(properties.getProperty("driver"));
         dataSource.setUrl(properties.getProperty("url"));
         dataSource.setUsername(properties.getProperty("username"));
         dataSource.setPassword(properties.getProperty("password"));
      } catch (IOException e) {
         e.printStackTrace();
      }
      return dataSource;
   }

   @Bean
   public JdbcTemplate jdbcTemplate() {
      JdbcTemplate template = new JdbcTemplate(mysqlDataSource());
      return template;
   }

   @Bean
   public PlatformTransactionManager txManager() {
```

```
        return new DataSourceTransactionManager(mysqlDataSource());
    }
}
```

EbillDAO.java

```
package com.cts.handson.dao;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Service;
import org.springframework.transaction.PlatformTransactionManager;
import org.springframework.transaction.TransactionStatus;
import org.springframework.transaction.support.DefaultTransactionDefinition;

import com.cts.handson.model.EBill;

@Service
@ComponentScan("com.cts.handson")
public class EBillDAO {
    @Autowired
    JdbcTemplate jdbcTemplate;
    @Autowired
    PlatformTransactionManager txManager;

    public void deleteBill(long...billNumbers) throws Exception {
```

```
        DefaultTransactionDefinition def = new DefaultTransactionDefinition();
        TransactionStatus status = txManager.getTransaction(def);
        try {
            for (long billNumber : billNumbers) {
                if (billNumber < 100) {
                    throw new Exception("Bill number is less than 100, rolling back transaction.");
                }
                String sql = "DELETE FROM EBILL WHERE BILL_NUMBER = ?";
                jdbcTemplate.update(sql, billNumber);
            }
            txManager.commit(status);
            System.out.println("Transaction committed successfully.");
        } catch(Exception ex) {
            txManager.rollback(status);
            System.out.println("Transaction rolled back due to error: " + ex.getMessage());
            throw ex;
        }
    }

    public List<EBill> getAllBill() {
        String sql = "SELECT BILL_NUMBER, CONSUMER_ID, BILLING_MONTH, READING, UNIT, AMOUNT FROM EBILL";
        return jdbcTemplate.query(sql, new BeanPropertyRowMapper<>(EBill.class));
    }
}
```

ElectricityBillApplication.java

```
package com.cts.handson;
```

```
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;
import org.springframework.context.annotation.ComponentScan;

import com.cts.handson.dao.EBillDAO;
import com.cts.handson.model.EBill;
import com.cts.handson.util.DateUtil;

@ComponentScan("com.cts.handson")
public class ElectricityBillApplication {
   public static void main(String[] args) {
      ApplicationContext appContext = new
AnnotationConfigApplicationContext(AppConfiguration.class);
      EBillDAO ebillDao = appContext.getBean(EBillDAO.class);

      try {
         // Delete records, including one with bill number < 100 to test rollback
         ebillDao.deleteBill(100L, 99L);
      } catch (Exception e) {
         // The exception is caught here to demonstrate the rollback
         System.out.println("Main method caught an exception: " + e.getMessage());
      }

      System.out.format("%-5s %-15s %-10s %10s %5s %s\n", "Id","Consumer
Id","Month","Reading","Unit","Amount");

      // Display all remaining bills
      ebillDao.getAllBill().forEach(System.out::println);
   }
}
```

Ebill.java

```
package com.cts.handson.model;


import java.util.Date;
import com.cts.handson.util.DateUtil;


public class EBill {
    private long billNumber;
    private String consumerId;
    private Date billingMonth;
    private long reading;
    private long unit;
    private double amount;
    public EBill(String consumerId, Date month, long reading, long unit, double amount)
{
            super();
            this.consumerId = consumerId;
            this.billingMonth = month;
            this.reading = reading;
            this.unit = unit;
            this.amount = amount;
    }
    public EBill() {
            super();
            // TODO Auto-generated constructor stub
    }
    public long getBillNumber() {
            return billNumber;
```

```
}
public void setBillNumber(long billNumber) {
	this.billNumber = billNumber;
}
public String getConsumerId() {
	return consumerId;
}
public void setConsumerId(String consumerId) {
	this.consumerId = consumerId;
}
public Date getBillingMonth() {
	return billingMonth;
}
public void setBillingMonth(Date billingMonth) {
	this.billingMonth = billingMonth;
}
public long getReading() {
	return reading;
}
public void setReading(long reading) {
	this.reading = reading;
}
public long getUnit() {
	return unit;
}
public void setUnit(long unit) {
	this.unit = unit;
}
public double getAmount() {
```

```
		return amount;
	}
	public void setAmount(double amount) {
		this.amount = amount;
	}
	@Override
	public String toString() {
		return String.format("%-5s %-15s %-10s %10s %5s %.2f",
billNumber,consumerId,DateUtil.dateToString(billingMonth),reading,unit,amount);
	}
}
```

DateUTIL.java

```
package com.cts.handson.util;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;

public class DateUtil {
	private final static SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy");
	public static Date stringToDate(String str) {
		Date date = null;
		try {
			date = sdf.parse(str);
		} catch (ParseException e) {
			date = new Date();
		}
```

```
            return date;
        }

        public static String dateToString(Date date) {
            return sdf.format(date);
        }
    }
```

ConnectionProperties.java

```
driver=com.mysql.cj.jdbc.Driver
url=jdbc:mysql://localhost:3306/egov
username=root
password=root
```

Maven

1) Build Web Application

Pom.xml

pom.xml

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>WebMaven</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>war</packaging>

    <name>WebMaven</name>
    <url>http://maven.apache.org</url>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
    </properties>

    <dependencies>
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>javax.servlet-api</artifactId>
            <version>3.1.0</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.8.1</version>
                <configuration>
                    <source>${maven.compiler.source}</source>
                    <target>${maven.compiler.target}</target>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.apache.tomcat.maven</groupId>
                <artifactId>tomcat7-maven-plugin</artifactId>
                <version>2.2</version>
                <configuration>
                    <port>9090</port>
                    <path>/WebMaven</path>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

2) Compiling Executing Java using POM

Main.java | pom.xml

```java
import java.io.*;
import java.util.*;
public class Main{
    public static void main(String args[])throws IOException{

        System.out.println("Maven Application");
    }
}
```

Main.java | pom.xml

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>MyMavenProject</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>MyMavenProject</name>
  <url>http://maven.apache.org</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.11</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-jar-plugin</artifactId>
        <version>3.2.0</version>
        <configuration>
          <archive>
            <manifest>
              <addClasspath>true</addClasspath>
              <mainClass>Main</mainClass>
            </manifest>
```

```xml
              <mainClass>Main</mainClass>
            </manifest>
          </archive>
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

3) Junit With Maven

pom.xml | CompanyBO.java | Main.java | CompanyJunit.java

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>Company</artifactId>
    <packaging>jar</packaging>
    <version>1.0-SNAPSHOT</version>
    <name>Company</name>
    <build>
        <sourceDirectory>src/main/java</sourceDirectory>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>2.22.2</version>
                <configuration>
                    <testSourceDirectory>src/test/java</testSourceDirectory>
                </configuration>
            </plugin>
        </plugins>
    </build>
    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.12</version>
            <exclusions>
                <exclusion>
                    <groupId>org.hamcrest</groupId>
                    <artifactId>hamcrest-core</artifactId>
                </exclusion>
            </exclusions>
        </dependency>
        <dependency>
            <groupId>org.hamcrest</groupId>
            <artifactId>hamcrest-library</artifactId>
            <version>1.3</version>
        </dependency>
    </dependencies>
```

4) Maven Directory Structure

Mvn archetype:generate -DgroupId=com.example – DartifaceId=EmployeeProject -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false

JUST COPY THIS CODE

6) Maven App to access External Service

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
   <modelVersion>4.0.0</modelVersion>

   <groupId>info.maigo.lab.hello</groupId>
   <artifactId>maigolab_hello</artifactId>
   <version>1.0-SNAPSHOT</version>
   <packaging>jar</packaging>

   <name>java-hello-world-with-maven</name>
```

```
    <url>http://maven.apache.org</url>

    <dependencies>
      <dependency>
        <groupId>com.mashape.unirest</groupId>
        <artifactId>unirest-java</artifactId>
        <version>1.4.9</version>
      </dependency>
      <dependency>
        <groupId>org.json</groupId>
        <artifactId>json</artifactId>
        <version>20170516</version>
      </dependency>
      <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.1</version>
        <scope>test</scope>
      </dependency>
    </dependencies>

    <build>
      <plugins>
        <plugin>
          <groupId>org.codehaus.mojo</groupId>
          <artifactId>exec-maven-plugin</artifactId>
          <version>1.6.0</version>
          <configuration>
            <mainClass>info.maigo.lab.hello.App</mainClass>
            <arguments>
              <argument>niwasawa</argument>
              <argument>maigolab</argument>
            </arguments>
          </configuration>
        </plugin>
      </plugins>
    </build>
</project>
```

App.java

```
package info.maigo.lab.hello;
```

```
import org.json.*;
import com.mashape.unirest.http.*;

public class App {

  public static void main(String[] args) throws Exception {
    for (String name: args) {
      System.out.println("name: " + name);
      String jsonString = new App(name).getUserInfoJSON();
      JSONObject json = new JSONObject(jsonString);
      System.out.println(json.toString(2));
    }
  }

  private final String name;

  public App(String name) {
    this.name = name;
  }

  public String getUserInfoURL() {
    // Users | GitHub Developer Guide: https://developer.github.com/v3/users/
    return "https://api.github.com/users/" + name;
  }

  public String getUserInfoJSON() throws Exception {
    String url = getUserInfoURL();
    HttpResponse<JsonNode> res = Unirest.get(url).asJson();
    return res.getBody().toString();
  }
}
```

Apptest.java

```
package info.maigo.lab.hello;

import junit.framework.Test;
import junit.framework.TestCase;
import junit.framework.TestSuite;

public class AppTest extends TestCase {

  public AppTest(String testName) {
```

```
    super(testName);
  }

  public static Test suite() {
    return new TestSuite( AppTest.class );
  }

  /**
   * getUserInfoURL Test
   */
  public void testGetUserInfoURL() {
    String name = "niwasawa";
    App app = new App(name);
    assertEquals("https://api.github.com/users/niwasawa", app.getUserInfoURL());
  }
}
```

7) Capital Service

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>capital-service</artifactId>
    <version>1.0-SNAPSHOT</version>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
    </properties>

    <dependencies>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.13.1</version>
            <scope>test</scope>
        </dependency>
```

```
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.8.1</version>
                <configuration>
                    <source>${maven.compiler.source}</source>
                    <target>${maven.compiler.target}</target>
                    <encoding>${project.build.sourceEncoding}</encoding>
                    <compilerArgs>
                        <arg>-Xlint:unchecked</arg>
                    </compilerArgs>
                </configuration>
            </plugin>
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>3.0.0</version>
                <executions>
                    <execution>
                        <phase>package</phase>
                        <goals>
                            <goal>java</goal>
                        </goals>
                    </execution>
                </executions>
                <configuration>
                    <mainClass>codes.CapitalService</mainClass>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```

Capitalservice.java

```
package codes;

import java.util.HashMap;
import java.util.Map;
```

```
public class CapitalService {

	public String greeting(String name)
	{
		return "Welcome "+name;
	}

	public String getCapital(String country)
	{
		Map<String, String> hashMap = new HashMap<>();
		hashMap.put("India", "New Delhi");
      hashMap.put("Singapore", "Singapore");
      hashMap.put("United Kingdom", "London");
      String capital = hashMap.get(country);
      if(capital!=null) return capital;
      else
        return "Country not found in database";
	}
	public static void main (String[] args)
	{
		System.out.println("Welcome to capital service");
		CapitalService cs=new CapitalService();
		System.out.println(cs.getCapital("India"));
	}
}
```

TestFramework.java

```
package mytest;
import codes.CapitalService;
import junit.framework.TestCase;

public class TestFramework extends TestCase
{
	public void testGreeting() {
		CapitalService cs=new CapitalService();
		assertEquals("New Delhi",cs.getCapital("India"));

	}
}
```